

Real-Time Mobile Video Streaming to TouchDesigner via WebRTC

Technical documentation and user guide

Coursework project — Internet and Mobile Systems II

Author: Jesse — Individual project

Abstract

This document describes the design and usage of a system that allows a mobile device, through a simple installation-free web application, to stream its camera feed in real time to TouchDesigner for creative video manipulation. The system is designed to work not only over a local network but also over the public Internet, which requires explicitly addressing typical mobile distributed-systems concerns: NAT traversal, signaling for establishing peer-to-peer connections, and interoperability between a web communication protocol (WebRTC) and a video transport protocol used in the creative and broadcast industry (NDI). The document presents the proposed architecture, the technology choices, the behaviour of each component, and a practical guide for installing, configuring and operating the system as a third party.

1. Introduction

1.1 Context and motivation

Real-time computer graphics tools such as TouchDesigner are widely used in performance, installation and audiovisual contexts to manipulate live images and video. A common limitation is the need for a video source physically wired to the computer running TouchDesigner (USB webcam, capture card, etc.), which reduces flexibility whenever a smartphone camera would be a more practical source — for example when it needs to be placed somewhere a cable cannot reach, or carried by a moving performer.

This project removes that constraint by turning any mobile device with a modern browser into a wireless video source for TouchDesigner, without requiring the installation of any dedicated app.

1.2 Objectives

- Capture the video feed from a mobile device's camera through a plain web page.
- Stream the feed to a TouchDesigner instance in real time, with low latency.
- Support operation even when the mobile device and the TouchDesigner machine are not on the same local network.
- Provide a documented, reusable architecture with a practical installation and usage guide for third parties.

2. Reference technologies

2.1 WebRTC

WebRTC (Web Real-Time Communication) is a standard natively supported by modern browsers that enables audio/video and data streaming directly between two peers, with built-in encoding and network handling. It is the technology chosen for capturing and transporting video from the mobile client, since it requires no plugins or native apps and automatically manages critical aspects such as adapting quality to available bandwidth.

2.2 Signaling and the WebSocket protocol

WebRTC defines how media is transported, but not how two peers exchange the initial information needed to establish a connection (SDP offers/answers, ICE candidates). This exchange, called signaling, is left to the application: in this project it is implemented over a WebSocket channel, which provides low-latency, bidirectional communication between the two endpoints.

2.3 NAT traversal: STUN and TURN

When client and server are not on the same local network, a direct connection is often blocked by routers and NAT (Network Address Translation). The ICE protocol uses STUN servers to discover a peer's public address and, when a direct connection is still not possible, TURN servers that relay media traffic. Since this project explicitly targets remote, Internet-based usage, having a reliable TURN server available is essential, not optional.

2.4 NDI (Network Device Interface)

NDI is a protocol developed by NewTek/Vizrt, widely adopted in the professional video and creative industry for transmitting high-quality, low-latency video streams over a local IP network. TouchDesigner natively supports receiving NDI streams through the NDI In TOP operator, making it the most natural format for bringing the video feed into the software.

2.5 TouchDesigner

TouchDesigner is a node-based visual development environment for building real-time interactive content, used in this project as the final destination of the video stream, where it can be manipulated with the software's native tools (TOPs, effects, compositing, etc.).

3. System architecture

3.1 Overview

The system consists of four main components, connected as shown in Figure 1. The core design decision is the separation between the signaling channel and the media channel: the former passes through the signaling server, while the latter travels directly between the client and the bridge following the WebRTC paradigm — peer-to-peer when possible, or via TURN relay when the network requires it.

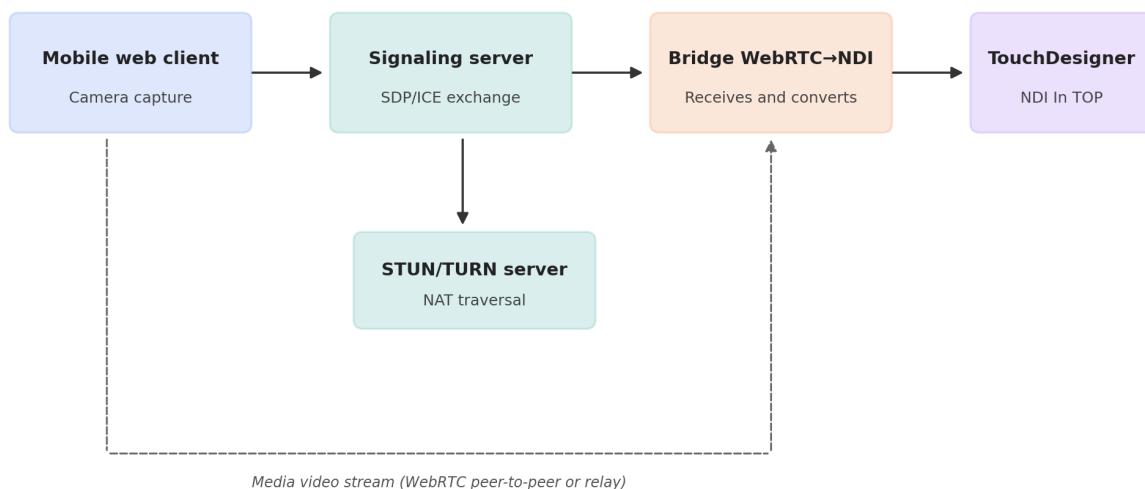


Figure 1 — System architecture: from the mobile client to TouchDesigner.

3.2 Components

Component	Role	Candidate technologies
Mobile web client	Captures the camera feed and sends it via WebRTC	HTML5, getUserMedia, RTCPeerConnection
Signaling server	Coordinates SDP/ICE exchange between client and bridge	Node.js, WebSocket (ws / Socket.IO)
STUN/TURN server	Enables NAT traversal and, when needed, relays media traffic	coturn

Component	Role	Candidate technologies
WebRTC → NDI bridge	Receives the WebRTC stream, decodes it and republishes it as an NDI source	Node.js (wrtc/mediasoup) or Python (aiortc) + NDI SDK
TouchDesigner	Receives the NDI stream and makes it available for creative manipulation	NDI In TOP

3.3 Operational flow

- The user opens the client web page's address from their mobile device's browser.
- The browser requests camera access; once granted, the feed is captured locally.
- The client opens a WebSocket connection to the signaling server and starts the WebRTC negotiation (SDP offer).
- The signaling server forwards the offer to the bridge, which responds with its own SDP answer and ICE candidates.
- A direct (or relayed) media connection is established between client and bridge via the STUN/TURN servers.
- The bridge receives the decoded video stream and republishes it on the local network as an NDI source.
- TouchDesigner, through an NDI In TOP operator, selects the source and makes it available for processing.

4. Implementation details

4.1 Mobile web client

The client is an HTML/JavaScript page that uses the `getUserMedia` API to access the camera and `RTCPeerConnection` to establish the WebRTC connection. The page must be served over HTTPS: mobile browsers deny camera access on insecure connections (the only exception being localhost addresses).

```
// simplified client excerpt
const stream = await navigator.mediaDevices.getUserMedia({
  video: { facingMode: 'environment' }, audio: false
});
const pc = new RTCPeerConnection({ iceServers: ICE_SERVERS });
stream.getTracks().forEach(track => pc.addTrack(track, stream));

const offer = await pc.createOffer();
await pc.setLocalDescription(offer);
signalingSocket.send(JSON.stringify({ type: 'offer', sdp: offer }));
```

4.2 Signaling server

The signaling server is a lightweight service that keeps WebSocket connections alive and forwards signaling messages (offers, answers, ICE candidates) between client and bridge, without ever processing the video content itself.

```
// simplified signaling server excerpt (Node.js)
const wss = new WebSocketServer({ port: 8443 });
wss.on('connection', (socket) => {
  socket.on('message', (msg) => {
    const data = JSON.parse(msg);
    forwardToPeer(data); // forward to the bridge or the client
  });
});
```

4.3 STUN/TURN server

For remote usage, installing a coturn server on a machine with a public IP address is recommended. A minimal configuration example:

```
# /etc/turnserver.conf
listening-port=3478
fingerprint
lt-cred-mech
user=username:password
realm=my-domain.example
cert=/etc/ssl/turn_cert.pem
pkey=/etc/ssl/turn_key.pem
```

4.4 WebRTC → NDI bridge

The bridge is the most delicate component: it must behave as a second WebRTC peer (accepting the client's offer), decode the incoming video frames, and pass them to a library that publishes them as an NDI source on the local network. The most practical combinations are a Node.js service based on mediasoup or wrtc together with NDI SDK bindings, or a Python service based on aiortc, both paired with the official NDI libraries for publishing the stream.

5. Installation guide

5.1 Prerequisites

- A computer with TouchDesigner installed and the NDI runtime/libraries installed.
- Node.js (LTS version) to run the signaling server and the bridge.
- A valid TLS certificate (e.g. via Let's Encrypt) to serve the client and signaling server over HTTPS/WSS.
- A machine with a public IP address for the STUN/TURN server, if remote access is required.

5.2 Setting up the STUN/TURN server

Install coturn (available in the repositories of most Linux distributions), configure it as shown in section 4.3, open the required UDP/TCP ports on the firewall (typically 3478 plus a relay range, e.g. 49152–65535), and start the service.

5.3 Starting the signaling server

```
npm install
node signaling-server.js
```

5.4 Starting the bridge

```
npm install
node bridge.js --ndi-name "Mobile camera"
```

5.5 Configuring TouchDesigner

- Create a new NDI In TOP operator in the TouchDesigner network.
- In the Source parameter, select the source published by the bridge (e.g. "Mobile camera").
- Check the Active parameter to confirm the stream is being received correctly.

6. End-user guide

6.1 Accessing from mobile

Open the browser on the mobile device (Safari on iOS, Chrome on Android) and navigate to the HTTPS address of the client page provided by the system's administrator/organiser.

6.2 Camera permissions

When prompted by the browser, grant camera access. On iOS Safari the permission is requested at every new session unless the page has been added to the Home Screen; on Android Chrome the permission can be made permanent for the domain.

6.3 Checking the connection

A visual indicator on the client page shows the connection status (connecting to signaling, negotiating, connected). Once the "connected" state is reached, the feed should appear a few moments later in TouchDesigner's NDI In TOP operator.

6.4 Troubleshooting

Symptom	Likely cause	Suggested action
The browser never asks for camera permission	Page not served over HTTPS	Check that the address starts with https://
Connection stuck on "negotiating"	STUN/TURN server unreachable	Check firewall ports and TURN credentials
NDI source not visible in TouchDesigner	Bridge not running or on a different network	Check that the bridge and TouchDesigner are on the same local network
High latency or choppy image	Insufficient bandwidth or overloaded TURN relay	Lower the resolution/bitrate on the client side

7. Security and privacy

Since the system transmits a potentially sensitive video stream over the Internet, a few precautions are recommended: exclusive use of HTTPS/WSS for all signaling channels, time-limited TURN

credentials rather than permanently shared ones, and a minimal authentication mechanism for accessing the client page (for example a per-session unique token), so that unauthorised parties cannot connect to the bridge.

8. Known limitations and future work

- Quality and latency depend heavily on the mobile network of the source device.
- The current design only sends video from the mobile device; a natural extension would add a data channel for remotely controlling TouchDesigner parameters.
- Supporting multiple simultaneous mobile clients requires extending the bridge to publish several NDI sources in parallel.

9. Conclusions

The system described here proposes a modular architecture able to connect a generic mobile device, through a plain web page, to a real-time video production environment such as TouchDesigner, overcoming the limits of wired connections and of local-network-only setups. The clean separation between the signaling channel and the media channel, combined with the use of open standards (WebRTC, NDI), makes the system extensible and reusable beyond its original context.

References

- W3C, "WebRTC: Real-Time Communication in Browsers", official specification.
- IETF RFC 8445, "Interactive Connectivity Establishment (ICE)".
- IETF RFC 5389, "Session Traversal Utilities for NAT (STUN)".
- IETF RFC 5766, "Traversal Using Relays around NAT (TURN)".
- NewTek/Vizrt, official NDI protocol documentation.
- Derivative, official TouchDesigner documentation — NDI In TOP operator.
- coturn, documentation of the open-source STUN/TURN server project.